

ARMENU ROADMAP

0 DAN 100 GACHA — BAJARILADIGAN YO'L XARITASI

AI yoki dasturchi shu hujjatni o'qib, loyihani boshidan oxirigacha to'xtamasdan qura oladi. Har vazifada: fayl yo'li, to'liq kod, tekshiruv.

Fazalar: 13 ta
Vazifalar: 234 ta (V-001 ... V-234)
Hajm: 39 650 qator
v1: 15 hafta · **v1.5:** 19,5 hafta
Sana: 2026-09-04

✔ **ROADMAP TO'LIQ.** Ishni boshlash uchun: `cat ROADMAP/START.md`
Bu PDF — boshlash qismi. Fazalar (37 617 qator) alohida .md fayllarda.

FAZALAR

13 faza · 234 vazifa

#	Faza	Vazifalar	Soni	Qator	Muddat
00	Tayyorgarlik	V-001…V-012	12	852	3 kun
01	Tenant + Auth	V-013…V-030	18	2303	1 hafta
02	Menyu	V-031…V-054	24	2714	2 hafta
03	Stol + QR	V-055…V-066	12	1180	1 hafta
04	Mijoz PWA	V-067…V-092	26	3293	2 hafta
05	Buyurtma + realtime	V-093…V-114	22	2996	2 hafta
06	⚡ AR/3D	V-115…V-142	28	4977	2,5 hafta
07	Zal plani	V-143…V-150	8	1589	1 hafta
08	Admin panel	V-151…V-170	20	4598	1,5 hafta
09	AI + MCP → v1 ✅	V-171…V-186	16	3420	1,5 hafta
10	To'lov + fiskal	V-187…V-204	18	3035	2 hafta
11	Sodiqlik + analitika	V-205…V-218	14	3160	1,5 hafta
12	Deploy → v1.5 ✅	V-219…V-234	16	3500	1 hafta

> Har faza alohida `.md` faylda: `ROADMAP/01_FAZALAR/FAZA_XX_....md`
> Har vazifada: **Fayl:** yo'li · to'liq ishlaydigan kod · ✅ Tekshirish buyrug'i. Placeholder yo'q.

01

Loyiha xaritasi

> Restoran stolidagi QR ni skanerlagan mijoz taomni telefon kamerasida **real o'lchamda 3D ko'radi**, buyurtma beradi; ofitsiant stol raqamini darhol ko'radi; admin hammasini real vaqtda nazorat qiladi.

🚀 ISHNI BOSHLASH

AI / dasturchi uchun — 1 ta buyruq

```
cat ROADMAP/START.md
```

Boshqa hech narsa o'qish shart emas. `START.md` sizni qadam-baqadam olib boradi:
holatni aniqlash → kontekst → muhit → faza → ishlash → tekshirish.

Odam uchun — loyihani tushunish

```
cat ROADMAP/00_BOSHLASH/01_LOYIHA_5_DAQIQADA.md
```

Yoki to'liq hujjat: **ARMENU_TZ_MASTER.pdf** (89 sahifa)

🗂 PAPKA STRUKTURASI

```
AR_MENU_LOYIHA/
│
├── README.md                    ← siz shu yerdasiz
│
├── ROADMAP/                     ⚡ BAJARILADIGAN — AI shu yerdan ishlaydi
│   ├── START.md                 ★ KIRISH NUQTASI — birinchi shu o'qiladi
│   │
│   ├── 00_BOSHLASH/
│   │   ├── 01_LOYIHA_5_DAQIQADA.md      loyiha nima (bir marta o'qiladi)
│   │   ├── 02_QOIDALAR.md               qanday kod yoziladi
│   │   └── 03_MUHIT_TAYYORLASH.md       nusxa-ko'chir buyruqlar
│   │
│   ├── 01_FAZALAR/              ⚡ 13 faza · 234 vazifa · 37 617 qator
│   │   ├── FAZA_00_TAYYORGARLIK.md       V-001…V-012  12 vazifa   852 q.
│   │   ├── FAZA_01_TENANT_AUTH.md        V-013…V-030  18 vazifa  2303 q.
│   │   ├── FAZA_02_MENYU.md              V-031…V-054  24 vazifa  2714 q.
│   │   ├── FAZA_03_STOL_QR.md            V-055…V-066  12 vazifa  1180 q.
│   │   ├── FAZA_04_MIJOZ_PWA.md          V-067…V-092  26 vazifa  3293 q.
│   │   ├── FAZA_05_BUYURTMA_REALTIME.md  V-093…V-114  22 vazifa  2996 q.
│   │   ├── FAZA_06_AR_3D.md         ⚡   V-115…V-142  28 vazifa  4977 q.
│   │   ├── FAZA_07_ZAL_PLAN.md           V-143…V-150   8 vazifa  1589 q.
│   │   ├── FAZA_08_ADMIN.md              V-151…V-170  20 vazifa  4598 q.
│   │   ├── FAZA_09_AI_MCP.md             V-171…V-186  16 vazifa  3420 q. → v1 ✅
│   │   ├── FAZA_10_TOLOV_FISKAL.md       V-187…V-204  18 vazifa  3035 q.
│   │   ├── FAZA_11_SODIQLIK.md           V-205…V-218  14 vazifa  3160 q.
│   │   └── FAZA_12_DEPLOY.md             V-219…V-234  16 vazifa  3500 q. → v1.5 ✅
│   │
│   ├── 02_MALUMOTNOMA/
│   │   ├── INDEKS.md            🔍 qaysi javob qaysi faylda (token tejash)
│   │   ├── TEZ_YORDAM.md        ⚡ takrorlanadigan kod naqshlari
│   │   └── TESTLAR.md           🧪 test strategiyasi, majburiy testlar
│   │
│   └── 03_HOLAT/
│       ├── PROGRESS.md          📊 qayerda to'xtaganini ko'rsatadi
│       ├── update_progress.py   🔧 avtomatik yangilash skripti
│       └── tekshir.sh           ✅ roadmap butunligini tekshiradi
│
├── TZ_MASTER/                   📚 MA'LUMOTNOMA — kerak bo'lganda o'qiladi
│   ├── ARMENU_TZ_MASTER.pdf     89 sahifa, to'liq hujjat
│   ├── 01_KIRISH_VA_BIZNES.md   biznes, bozor, investor
│   ├── 02_SAHIFALAR_VA_TUGMALAR.md  har sahifa, har tugma
│   ├── 03_ARXITEKTURA.md        masshtablash, chidamlilik
│   ├── 04_DATA_MODEL_VA_API.md  DB, API, WebSocket
│   ├── 05_KOD_YOZISH_TARTIBI.md fayl strukturasi
│   ├── 06_TARJIMA_VA_UI.md      3 til, dizayn
│   ├── 07_AI_MCP.md             8 AI provayder
│   └── 08_DEPLOY_REJA_RISK.md   deploy, risk, savollar
│
├── BOZOR_TAHLIL.md              bozor: fiskalizatsiya, delivery, turizm
├── _ARXIV/                      eski versiyalar
└── ai agent.mp4                 foydalanuvchi bergan video
```

📊 HOZIRGI HOLAT

```
# Qayerda to'xtaganini ko'rish
python3 ROADMAP/03_HOLAT/update_progress.py --status

# Roadmap butunligini tekshirish
bash ROADMAP/03_HOLAT/tekshir.sh
```

Roadmap	✅ TO'LIQ — 13 faza, 234 vazifa, 39 650 qator
Kod	🔴 Boshlanmagan — ruxsat kutilmoqda
Keyingi vazifa	V-001 (muhit tayyorlash)
v1 muddat	15 hafta (186 vazifa)
v1.5 muddat	19,5 hafta (234 vazifa)

⚡ ASOSIY FAKTLAR

Stack	Django 5 + DRF + Channels · PostgreSQL + pgvector · Redis
Frontend	PWA (ilova YO'Q) · Telegram Mini App
AR	`<model-viewer>` → iOS: USDZ · Android: GLB
3 rol	Mijoz (QR) · Ofitsiant (PIN) · Admin (login)
Tillar	O'zbek · Rus · Ingliz (100% qamrov)
AI	8 provayder MCP orqali (fallback zanjiri)
To'lov	Payme · Click · Uzum (`paytechuz`)
Fiskal	UZKASSA · e-POS (OFD) + IKPU
Port	web `8060` · ws `8061`
DB	`armenu_db` · Redis `db=5/6`

⛔ IZOLYATSIYA (o'zgarmas qoida)

Bu **alohida loyiha**. `mebelcity_platform` ga hech qanday tegish yo'q:

	ARMENU	mebelcity_platform
Papka	`/home/user/armenu/`	TEGILMAYDI
DB	`armenu_db`	`tenant_*` TEGILMAYDI
Redis	`db=5` , `db=6`	`db=0..4` TEGILMAYDI
Port	8060, 8061	8040, 8041 TEGILMAYDI
Servis	`armenu*.service`	`bittada-*` TEGILMAYDI

🎯 BOZOR (qisqacha)

O'zbekistonda ovqatlanish korxonalari	**27 312**
Sektor daromadi (I chorak 2026)	**49,8 trln so'm** (+8,2%)
Xalqaro turistlar (7 oy 2026)	**7,75 mln** (+22,6%)
AR/3D raqobatchi	**0 ta** ✅
Break-even	11-oy
3-yil prognoz	~2,6 mlrd so'm daromad

Batafsil: BOZOR_TAHLIL.md

? KOD BOSHLASHDAN OLDIN

12 ta savolga javob kerak — TZ_MASTER/08_DEPLOY_REJA_RISK.md §7.

Eng muhim 5 tasi:
1. Server: alohida VPS? *(tavsiya: ha)*
2. Domen va brend nomi?
3. Pilot restoran bormi? *(5 ta kerak)*
4. Kim yozadi? *(2 dasturchi — 15 hafta)*
5. AI API kalitlari bormi? *(kamida 3 ta)*

02

START — kirish nuqtasi

> **AI/dasturchi uchun:** Bu loyihaning **yagona kirish nuqtasi**.
> Boshqa hech qanday faylni oldin o'qima. Quyidagi 6 qadamni bajar.

⚡ QADAM 1 — HOLATNI ANIQLA (30 soniya)

```
cat ROADMAP/03_HOLAT/PROGRESS.md
```

Bu faylda **oxirgi bajarilgan vazifa raqami** yozilgan. Masalan:

```
OXIRGI BAJARILGAN: V-042
KEYINGI: V-043
FAZA: 2 (Menyu)
```

Agar `PROGRESS.md` **da** `V-000` **bo'lsa** → loyiha hali boshlanmagan, QADAM 2 ga o't.
Agar boshqa raqam bo'lsa → QADAM 4 ga sakra (davom ettirish).

⚡ QADAM 2 — 15 DAQIQALIK KONTEKST (faqat bir marta)

Loyihani tushunish uchun **faqat shu 2 ta faylni** o'qi (jami ~12 KB):

```
cat ROADMAP/00_BOSHLASH/01_LOYIHA_5_DAQIQADA.md    # nima quriladi
cat ROADMAP/00_BOSHLASH/02_QOIDALAR.md             # qanday kod yoziladi
```

> ⛔ `TZ_MASTER/` **papkasidagi fayllarni HOZIR O'QIMA.**
> Ular ma'lumotnoma — faqat kerak bo'lganda, kerakli bo'limini o'qiysan.
> Hammasini o'qish = 260 KB = ~70 000 token behuda sarflanadi.

⚡ QADAM 3 — MUHITNI TAYYORLA

```
cat ROADMAP/00_BOSHLASH/03_MUHIT_TAYYORLASH.md
```

Bu faylda **nusxa ko'chirib ishga tushiriladigan buyruqlar** bor.
Bajar, natijani tekshir, keyin QADAM 4 ga o't.

⚡ QADAM 4 — FAZANI OCH VA ISHLA

Hozirgi fazani `PROGRESS.md` dan bilib olding. Shu faza faylini och:

Faza	Fayl	Vazifalar	Muddat
0	`01_FAZALAR/FAZA_00_TAYYORGARLIK.md`	V-001...V-012	3 kun
1	`01_FAZALAR/FAZA_01_TENANT_AUTH.md`	V-013...V-030	1 hafta
2	`01_FAZALAR/FAZA_02_MENYU.md`	V-031...V-054	2 hafta
3	`01_FAZALAR/FAZA_03_STOL_QR.md`	V-055...V-066	1 hafta
4	`01_FAZALAR/FAZA_04_MIJOZ_PWA.md`	V-067...V-092	2 hafta
5	`01_FAZALAR/FAZA_05_BUYURTMA_REALTIME.md`	V-093...V-114	2 hafta
6	`01_FAZALAR/FAZA_06_AR_3D.md` ⚡	V-115...V-142	2,5 hafta
7	`01_FAZALAR/FAZA_07_ZAL_PLAN.md`	V-143...V-150	1 hafta
8	`01_FAZALAR/FAZA_08_ADMIN.md`	V-151...V-170	1,5 hafta
9	`01_FAZALAR/FAZA_09_AI_MCP.md`	V-171...V-186	1,5 hafta
—	**v1 TUGADI**		**15 hafta**
10	`01_FAZALAR/FAZA_10_TOLOV_FISKAL.md`	V-187...V-204	2 hafta
11	`01_FAZALAR/FAZA_11_SODIQLIK.md`	V-205...V-218	1,5 hafta
12	`01_FAZALAR/FAZA_12_DEPLOY.md`	V-219...V-234	1 hafta
—	**v1.5 TUGADI**		**19,5 hafta**

Faza faylida **har bir vazifa uchun**:
- 📄 Qaysi fayl yaratiladi
- 💻 Nima kod yoziladi (namuna bilan)
- ✅ Qanday tekshiriladi
- ➡ Keyingi qadam

⚡ QADAM 5 — HAR VAZIFADAN KEYIN

```
# 1. Testni ishga tushir
pytest tests/ -x -q

# 2. Progress'ni yangila
python3 ROADMAP/03_HOLAT/update_progress.py V-043 ok

# 3. Commit
git add -A && git commit -m "V-043: menyu API keshi"
```

⚠ **Test o'tmasa keyingi vazifaga O'TMA.** Tuzat, keyin davom et.

⚡ QADAM 6 — FAZA TUGAGANDA

```
cat ROADMAP/01_FAZALAR/FAZA_XX_....md   # oxiridagi "FAZA QABULI" bo'limi
```

U yerda faza qabul mezonlari bor. Hammasi ✅ bo'lsa — keyingi fazaga o't.

🕮 MA'LUMOTNOMA — QACHON NIMANI O'QISH

> ⚡ **Token tejash qoidasi:** faqat KERAK bo'lganda, faqat KERAKLI bo'limini o'qi.

Savol tug'ilsa	O'qi
"Bu tugma nima qiladi?"	`TZ_MASTER/02_SAHIFALAR_VA_TUGMALAR.md` — kerakli sahifa (M-xx / O-xx / A-xx)
"Bu jadvalda qanday maydonlar?"	`TZ_MASTER/04_DATA_MODEL_VA_API.md` §1
"API javob formati qanday?"	`TZ_MASTER/04_DATA_MODEL_VA_API.md` §2
"WebSocket xabar turi qanday?"	`TZ_MASTER/04_DATA_MODEL_VA_API.md` §3
"Masshtablash qanday?"	`TZ_MASTER/03_ARXITEKTURA.md` §4
"Xato bo'lsa nima qilinadi?"	`TZ_MASTER/03_ARXITEKTURA.md` §5
"Rang/shrift qanday?"	`TZ_MASTER/06_TARJIMA_VA_UI.md` §2
"Tarjima qanday ishlaydi?"	`TZ_MASTER/06_TARJIMA_VA_UI.md` §1
"AI provayder qanday qo'shiladi?"	`TZ_MASTER/07_AI_MCP.md` §4
"Nima uchun bu qaror qabul qilingan?"	`TZ_MASTER/01_KIRISH_VA_BIZNES.md`
"Deploy qanday?"	`TZ_MASTER/08_DEPLOY_REJA_RISK.md` §1

Tez qidiruv:

```
grep -n "izlagan_narsangiz" ROADMAP/02_MALUMOTNOMA/INDEKS.md
```

⛔ QAT'IY QOIDALAR (buzilmaydi)

#	Qoida
1	⛔ `mebelcity_platform` **ga TEGMA** — alohida repo, DB, port, servis
2	⛔ **Vazifa tartibini buzma** — V-043 dan oldin V-042 bajarilishi shart
3	⛔ **Test o'tmasa davom etma**
4	⛔ **TZ_MASTER ni to'liq o'qima** — faqat kerakli bo'limni
5	✅ **Har vazifadan keyin PROGRESS.md yangilanadi**
6	✅ **Har model tenant-scoped** (`restaurant_id` filtri)
7	✅ **Har UI 3 tilda + dark/light**
8	✅ **Miqdor butun son** (`Math.ceil()`)
9	✅ **Narx serverda hisoblanadi** — frontendga ishonilmaydi
10	✅ **Uzoq ish → worker**, web so'rovda emas

🔄 ISH SIKLI (to'xtamasdan ishlash uchun)

> Bu bo'lim AI uchun. Har vazifada **aynan shu 6 qadam** takrorlanadi.
> Boshqa hech narsa o'ylab topish shart emas.

```
┌─────────────────────────────────────────────────────┐
│  1. PROGRESS.md dan keyingi vazifa raqamini ol   (V-XXX)     │
│     $ python3 ROADMAP/03_HOLAT/update_progress.py --status   │
├─────────────────────────────────────────────────────┤
│  2. Shu vazifa turgan faza faylini och                       │
│     $ grep -A 60 "## V-XXX" ROADMAP/01_FAZALAR/FAZA_YY_*.md  │
│     (butun faylni emas — faqat shu vazifani)                 │
├─────────────────────────────────────────────────────┤
│  3. Kodni yoz (fayl yo'li va kod vazifada berilgan)          │
│     Savol tug'ilsa → ROADMAP/02_MALUMOTNOMA/INDEKS.md        │
├─────────────────────────────────────────────────────┤
│  4. Tekshir                                                  │
│     $ pytest tests/ -x -q                                    │
│     ✗ o'tmadi → TUZAT, 3-qadamga qayt                        │
│     ✓ o'tdi   → 5-qadam                                      │
├─────────────────────────────────────────────────────┤
│  5. Yozib qo'y                                               │
│     $ git add -A && git commit -m "V-XXX: <tavsif>"          │
│     $ python3 ROADMAP/03_HOLAT/update_progress.py V-XXX ok   │
├─────────────────────────────────────────────────────┤
│  6. 1-qadamga qayt  ← TO'XTAMA, keyingi vazifaga o't         │
│     V-234 tugaguncha yoki foydalanuvchi to'xtatguncha        │
└─────────────────────────────────────────────────────┘
```

⛔ Qachon TO'XTASH kerak

Holat	Nima qilish
Test 3 marta ketma-ket o'tmadi	To'xta, muammoni PROGRESS.md ga yoz, foydalanuvchiga ayt
Vazifada API kaliti kerak, lekin `.env` bo'sh	Shu vazifani `skip` qil, keyingisiga o't, oxirida ayt
Vazifa TZ da noaniq	INDEKS orqali qidir; topilmasa — eng oddiy variantni tanla va PROGRESS ga yoz
Faza tugadi	Faza qabul checklistini tekshir, keyin keyingi fazaga o't
V-234 tugadi	🎉 To'xta, hisobot ber

⚡ TOKEN TEJASH (majburiy)

✖ Qilma	✔ Qil
`cat TZ_MASTER/*.md` (260 KB)	`grep -A 45 "class Order" TZ_MASTER/04_*.md` (2 KB)
Butun faza faylini o'qish	`grep -A 60 "## V-043" FAZA_02_MENYU.md`
Har vazifada qoidalarni qayta o'qish	Bir marta o'qib, yodda tut
Yozgan kodni qayta o'qish	Edit natijasiga ishon
Bir necha vazifani bitta commitga	1 vazifa = 1 commit (kontekst toza)

Taxminiy token sarfi:

```
START.md + qoidalar (bir marta)       ~ 6 000 token
Har vazifa (grep + kod yozish)        ~ 3 000 token
234 vazifa                            ~700 000 token
```

🎯 LOYIHA MAQSADI (1 jumla)

> Restoran stolidagi QR ni skanerlagan mijoz taomni telefon kamerasida **real o'lchamda 3D ko'radi**, buyurtma beradi; ofitsiant stol raqamini darhol ko'radi; admin hammasini real vaqtda nazorat qiladi.

Hozir nima qilish kerak: ⬆ QADAM 1 dan boshla.

0 3

Loyiha 5 daqiqada

> Bu faylni **bir marta** o'qi. Loyihani tushunish uchun shu yetarli.

---

1. NIMA QURILADI

ARMENU — restoran/choyxona uchun AR/3D menyu platformasi.

```
Mijoz stolga o'tiradi
   │
   ├─ Stoldagi QR ni skanerlaydi  ──► telefonida menyu ochiladi (3 tilda)
   │
   ├─ Taomni bosadi ──► "AR'da ko'rish" ──► kamera ochiladi
   │                                        taom stol ustida REAL O'LCHAMDA
   │                                        paydo bo'ladi, aylantiriladi
   │
   ├─ Yoqdi ──► savatga ──► buyurtma
   │
   ▼
Ofitsiant telefoniga: 🔔 "STOL 5 — yangi buyurtma"
Oshxona monitoriga:   buyurtma kanban taxtasiga tushadi
Admin ekraniga:       jonli oqimda ko'rinadi
   │
   ▼
Oshpaz tayyorlaydi ──► "Tayyor" ──► ofitsiant olib boradi ──► mijoz to'laydi
```

---

2. 3 ROL

Rol	Kirish	Nima qiladi
🧑 **Mijoz**	QR skanerlash (auth yo'q)	Menyu ko'radi, AR'da ko'radi, buyurtma beradi, to'laydi
👨‍🍳 **Ofitsiant**	PIN kod	Buyurtmani ko'radi, **stol raqamini** biladi, olib boradi
👨‍💼 **Admin**	Login	Menyu, stol, xodim, hisobot — hammasini boshqaradi

Qo'shimcha: 👨‍🍳 **KDS** (oshxona TV monitori), 💵 **Kassa**, 🔧 **Super-admin** (platforma egasi).

---

3. TEXNOLOGIYALAR

```
Backend    Django 5 + DRF + Channels (WebSocket)
DB         PostgreSQL (+ pgvector) + Redis
Frontend   PWA (ilova YO'Q) — vanilla JS / Vue 3
AR/3D      <model-viewer>  →  iOS: USDZ (AR Quick Look)
                              Android: GLB (Scene Viewer)
3D yaratish Meshy / Tripo API (rasm → 3D)
To'lov     paytechuz (Payme + Click + Uzum)
Fiskal     UZKASSA / e-POS (OFD)
AI         8 provayder MCP orqali (Claude, Gemini, GPT, Groq, ...)
Deploy     systemd + nginx + gunicorn + daphne
```

---

4. ⚡ ENG MUHIM 5 TA G'OYA

4.1 AR real o'lchamda bo'lishi SHART

Model **metrda** modellanadi (lagan Ø32 sm = `0.32 m`), `ar-scale="fixed"`.
Noto'g'ri o'lcham = butun loyiha ma'nosini yo'qotadi.

4.2 3D kutubxona — asosiy aktiv

Umumiy o'zbek taomlari bazasi (osh, manti, somsa...). Yangi restoran kelsa — model **tayyor**, 0 xarajat. Bu raqobatchi 2-3 yilda takrorlay olmaydigan ustunlik.

```
1-restoran:   kontent 1 250 000 so'm  →  marja 62%
100-restoran: kontent    50 000 so'm  →  marja 85%
```

4.3 Multi-tenant izolyatsiya — xavfsizlik #1

Har so'rovda `restaurant_id` filtri. Restoran A restoran B ma'lumotini **hech qachon** ko'rmasligi kerak. CI'da test majburiy.

4.4 Mijozga WebSocket faqat buyurtmadan keyin

Menyu ko'rayotgan mijozga WS **kerak emas**. Bu 1 mln foydalanuvchida WS ulanishni 1 mln dan ~90 000 ga tushiradi (20 server o'rniga 3 ta).

4.5 Uzoq ish → worker servis

3D generatsiya 3-40 daqiqa. Web so'rovda **hech qachon** qilinmaydi. Redis navbat → alohida worker servis.

---

5. FAZALAR (0 → 100)

```
FAZA 0   Tayyorgarlik        3 kun     Django skelet ishlaydi
FAZA 1   Tenant + Auth       1 hafta   Restoran, xodim, PIN kirish
FAZA 2   Menyu               2 hafta   Kategoriya, taom, 3 til, kesh
FAZA 3   Stol + QR           1 hafta   Noyob QR, PDF chop
FAZA 4   Mijoz PWA           2 hafta   Menyu ko'rish, savat
FAZA 5   Buyurtma + realtime 2 hafta   Buyurtma, KDS, ofitsiant
FAZA 6   ⚡ AR/3D           2,5 hafta  AR ISHLAYDI ← asosiy
FAZA 7   Zal plani           1 hafta   2D drag&drop
FAZA 8   Admin panel        1,5 hafta  Dashboard, hisobot
FAZA 9   AI + MCP           1,5 hafta  Chatbot, avto-tarjima
─────────────────────────────────────────────────────
                       v1 = 15 hafta  ✅ ISHLAYDIGAN MAHSULOT
FAZA 10  To'lov + fiskal     2 hafta   Payme/Click, OFD chek
FAZA 11  Sodiqlik + analitika 1,5 hafta Keshbek, otziv, menyu-injiniring
FAZA 12  Deploy + kuzatuv    1 hafta   Production, monitoring
─────────────────────────────────────────────────────
                     v1.5 = 19,5 hafta ✅ SOTUVGA TAYYOR
```

---

6. ⛔ IZOLYATSIYA (eng muhim ogohlantirish)

Bu **alohida loyiha**. `mebelcity_platform` ga hech qanday tegish yo'q:

	ARMENU	mebelcity_platform
Papka	`/home/user/armenu/`	TEGILMAYDI
DB	`armenu_db`	`tenant_*` TEGILMAYDI
Redis	`db=5`	`db=0..4` TEGILMAYDI
Port	8060 (web), 8061 (ws)	8040/8041 TEGILMAYDI
Servis	`armenu*.service`	`bittada-*` TEGILMAYDI

---

7. FAYL STRUKTURASI (qisqacha)

```
armenu/
├── config/settings/{base,dev,prod,test}.py
├── core/           ← umumiy: middleware, mixin, resilience, queue, ws
├── apps/
│   ├── tenants/    ← Restaurant (asosiy tenant)
│   ├── accounts/   ← User, Staff, JWT, PIN
│   ├── menu/       ← Category, Dish, Modifier
│   ├── tables/     ← Hall, Table, QR
│   ├── guest/      ← Guest, GuestSession
│   ├── orders/     ← Order, holat mashinasi, KDS
│   ├── ar3d/       ← ⚡ DishModel3D, DishLibrary, pipeline
│   ├── payments/   ← Payme, Click, Uzum
│   ├── fiscal/     ← OFD, IKPU
│   ├── ai/         ← ⚡ MCP router, chatbot
│   └── ...
├── static/{guest,staff,kds,admin_ui}/
├── templates/
├── locale/{uz,ru,en}/
└── tests/
```

04

Qoidalar

> Bu faylni **bir marta** o'qi va butun loyiha davomida shu qoidalarga amal qil.

---

1. ⛔ 10 TA BUZILMAS QOIDA

#	Qoida	Nima uchun
1	`mebelcity_platform` **ga tegma**	Alohida loyiha
2	**Vazifa tartibini buzma** (V-042 → V-043)	Bog'liqlik
3	**Test o'tmasa davom etma**	Sifat
4	**Har tenant modeli** `restaurant_id` **bilan filtrlanadi**	Xavfsizlik #1
5	**Narx faqat serverda hisoblanadi**	Frontendga ishonilmaydi
6	**Miqdor butun son** — `Math.ceil()`	Biznes qoidasi
7	**Uzoq ish (>2s) → worker navbat**	Web bloklanmaydi
8	**Har UI 3 tilda + dark/light**	Talab
9	**Har yozuv operatsiyasi idempotent**	Retry xavfsiz
10	**Tashqi API → circuit breaker + fallback**	Chidamlilik

---

2. 🔒 MULTI-TENANT (eng muhim)

✅ TO'G'RI

```
# View'da mixin ishlatiladi
class DishViewSet(TenantScopedMixin, ModelViewSet):
    queryset = Dish.objects.all()      # mixin avtomatik filtrlaydi

# Yoki qo'lda — DOIM restaurant_id bilan
dish = Dish.objects.get(pk=pk, restaurant_id=request.restaurant_id)
```

❌ NOTO'G'RI

```
dish = Dish.objects.get(pk=pk)         # ❌ boshqa restoranniki bo'lishi mumkin!
dishes = Dish.objects.all()            # ❌ hamma restoran taomlari!
```

Har yangi model uchun

```
class YangiModel(TenantModel):          # TenantModel = restaurant FK bor
    ...
```

Va `tests/test_tenant_isolation.py` ga qo'shiladi.

---

3. 🗄 DATABASE QOIDALARI

N+1 query oldini olish

```
# ❌ NOTO'G'RI — har buyurtma uchun alohida so'rov
for order in Order.objects.all():
    print(order.table.number)

# ✅ TO'G'RI
for order in Order.objects.select_related('table', 'guest'):
    print(order.table.number)

# ✅ M2M / teskari FK uchun
Order.objects.prefetch_related('items__dish')
```

Katta ma'lumot

```
# ✗ list(qs) — xotiraga hammasi
# ✓ qs.iterator(chunk_size=500)
```

Transaksiya

```
with transaction.atomic():
    order = Order.objects.create(...)
    OrderItem.objects.bulk_create(items)     # bitta so'rov

# Post-commit hook — transaksiya MUVAFFAQIYATLI bo'lgandan keyin
transaction.on_commit(lambda: send_ws_notification(order))
```

Indeks

Har `filter()` da ishlatiladigan maydonga indeks:

```
class Meta:
    indexes = [
        models.Index(fields=['restaurant', 'status', '-created_at']),
    ]
```

---

4. 🌐 API QOIDALARI

Javob formati

```
# Muvaffaqiyat
return Response({'id': 1, 'name': '...'}, status=200)

# Xato — DOIM 3 tilda
raise DishUnavailable(dish)     # core/exceptions.py handler formatlaydi
```

Idempotency (POST/PUT)

```
idem_key = request.headers.get('X-Idempotency-Key')
existing = check_idempotency(idem_key)
if existing:
    return Response(existing)        # dublikat yaratilmaydi
```

Validatsiya tartibi (qat'iy)

```
1. Idempotency
2. Tenant tekshiruvi (restaurant_id)
3. Obyekt mavjudligi
4. Biznes qoidalari (stop-list, limit)
5. ⚡ Narx serverda qayta hisoblanadi
6. Transaksiya
7. Post-commit hooks (WS, navbat)
```

---

5. ⚡ REAL-TIME (WebSocket)

Guruh nomlari — DOIM restoran-scoped

```
ws_send(f'rest_{restaurant_id}_kds',     'order.new', data)
ws_send(f'rest_{restaurant_id}_waiters', 'order.new', data)
ws_send(f'guest_{session_id}',           'order.created', data)

# ✗ HECH QACHON global guruh: ws_send('all_orders', ...)
```

Xabar formati

```
{'type': 'order.new', 'ts': '...', 'data': {...}, 'seq': 1047}
```

Mijozga WS qachon

```
✗ Menyu ko'rayotganda — KERAK EMAS
✓ Buyurtma bergandan KEYIN — kerak
```

---

6. 🎨 FRONTEND QOIDALARI

Tarjima

```
// ✗ NOTO'G'RI
btn.textContent = "Savatga qo'shish";

// ✓ TO'G'RI
btn.textContent = t('dish.add_to_cart');
// yoki HTML'da: <button data-i18n="dish.add_to_cart"></button>
```

Tema

```
/* ✗ NOTO'G'RI */
.card { background: #fff; color: #000; }

/* ✓ TO'G'RI */
.card { background: var(--bg-card); color: var(--text); }
```

Miqdor

```
// ✓ DOIM
const qty = Math.max(1, Math.min(99, Math.ceil(rawQty)));
```

Statik fayl versiyasi

```
<script src="/static/guest/js/menu.js?v=12"></script>
```

---

7. 🧪 TEST QOIDALARI

Har vazifadan keyin

```
pytest tests/ -x -q          # -x = birinchi xatoda to'xtaydi
```

Har yangi model uchun majburiy test

```
def test_<model>_tenant_isolation(api_client, restaurant, restaurant2, token):
    obj = <Model>Factory(restaurant=restaurant2)
    r = api_client.get(f'/v1/admin/<endpoint>/{obj.id}/',
                       HTTP_AUTHORIZATION=f'Bearer {token}')
    assert r.status_code in (403, 404)      # 200 BO'LMASLIGI SHART
```

Qamrov

```
Umumiy ≥ 75%
orders, payments, tenants ≥ 90%
```

---

8. 📁 FAYL VA NOM QOIDALARI

Nima	Uslub	Namuna
Python fayl	snake_case	`order_service.py`
Django app	lowercase	`apps/ar3d/`
Model	PascalCase	`DishModel3D`
Funksiya	snake_case	`create_order()`
Konstanta	UPPER_SNAKE	`MAX_GLB_SIZE`
JS fayl	kebab-case	`ar-detect.js`
CSS klass	kebab-case	`.dish-card`
i18n kalit	dot.case	`dish.add_to_cart`
API endpoint	kebab-case	`/v1/admin/qr-pdf/`
WS xabar	dot.case	`order.new`

---

9. 💬 COMMIT QOIDALARI

```
git commit -m "V-043: menyu API Redis keshi qo'shildi"
git commit -m "V-115: DishLibrary modeli + pgvector indeks"
git commit -m "fix V-098: idempotency key tekshiruvi tuzatildi"
```

Format: `V-XXX: <qisqa tavsif>` yoki `fix V-XXX: <nima tuzatildi>`

---

10. 🚨 XATO QAYTA ISHLASH

Har tashqi chaqiruv

```
from core.resilience import CircuitBreaker

breaker = CircuitBreaker('ofd', fail_threshold=3, timeout=120)

result = breaker.call(
    ofd_api.send, receipt,
    fallback=lambda: enqueue('fiscal', receipt.id, delay=300)
)
```

Log darajalari

```
log.debug()    # ishlab chiqishda
log.info()     # muhim hodisa (buyurtma yaratildi)
log.warning()  # muammo, lekin ishlaydi (provayder yiqildi, fallback)
log.error()    # xato, aralashuv kerak
log.exception()# xato + stack trace
```

---

11. ⚡ TOKEN TEJASH (AI uchun)

Bu loyihada ishlayotgan AI uchun:

Qoida	Nima uchun
`TZ_MASTER/` ni **to'liq o'qima**	260 KB = ~70k token behuda
Faqat faza faylini o'qi	~15 KB = ~4k token
Ma'lumotnomadan **faqat kerakli §** ni	`sed -n '120,180p' fayl.md`
Kod yozishdan oldin mavjud faylni o'qi	Takrorlamaslik uchun
`grep` bilan qidir, butun fayl o'qima	Tez va arzon
Bir vazifa = bir commit	Kontekst toza qoladi

Namuna — to'g'ri qidirish:

```
# ✗ cat TZ_MASTER/04_DATA_MODEL_VA_API.md          (36 KB)
# ✓ grep -A 40 "class Order" TZ_MASTER/04_DATA_MODEL_VA_API.md   (2 KB)
```

---

12. ✅ "TUGALLANGAN" TA'RIFI

Vazifa faqat quyidagilar bajarilgandagina tugallangan:

```
[ ] Kod yozilgan va ishlaydi
[ ] Test yozilgan va o'tadi
[ ] Tenant izolyatsiyasi tekshirilgan (tenant modeli bo'lsa)
[ ] 3 til (UI bo'lsa)
[ ] Dark + Light (UI bo'lsa)
[ ] Xato holatlari qayta ishlangan
[ ] PROGRESS.md yangilangan
[ ] Commit qilingan
```

05

Muhit tayyorlash

> Buyruqlarni **ketma-ket** bajar. Har blokdan keyin tekshiruvni ishga tushir.

---

0. OLDINDAN KERAK

Nima	Versiya	Tekshirish
Python	3.12+	`python3 --version`
PostgreSQL	15+	`psql --version`
Redis	7+	`redis-server --version`
Node.js	20+	`node --version` (3D optimizatsiya uchun)
Git	—	`git --version`

Yo'q bo'lsa:

```
sudo apt update
sudo apt install -y python3.12 python3.12-venv python3-pip \
     postgresql postgresql-contrib redis-server git nodejs npm \
     libpq-dev build-essential
```

---

1. LOYIHA PAPKASI

```
# ⚠ mebelcity_platform DAN TASHQARIDA
mkdir -p /home/user/armenu
cd /home/user/armenu
git init
```

Tekshirish: `pwd` → `/home/user/armenu`

---

2. VIRTUAL MUHIT

```
cd /home/user/armenu
python3.12 -m venv venv
source venv/bin/activate
python -m pip install --upgrade pip
```

Tekshirish: `which python` → `/home/user/armenu/venv/bin/python`

---

3. PAKETLAR

```
cat > requirements.txt <<'EOF'
Django==5.0.9
djangorestframework==3.15.2
channels[daphne]==4.1.0
channels-redis==4.2.0
psycopg[binary]==3.2.3
redis==5.1.1
django-redis==5.4.0
django-cors-headers==4.4.0
drf-spectacular==0.27.2
python-decouple==3.8
Pillow==10.4.0
qrcode[pil]==7.4.2
reportlab==4.2.2
pgvector==0.3.3
gunicorn==22.0.0
sentry-sdk==2.14.0
paytechuz==0.4.0
requests==2.32.3
anthropic==0.40.0
google-genai==0.3.0
openai==1.54.0
groq==0.11.0
pygltflib==1.16.2
python-slugify==8.0.4
bcrypt==4.2.0
PyJWT==2.9.0
pyotp==2.9.0
EOF

cat > requirements-dev.txt <<'EOF'
pytest==8.3.3
pytest-django==4.9.0
pytest-cov==5.0.0
pytest-asyncio==0.24.0
factory-boy==3.3.1
ruff==0.7.0
black==24.10.0
bandit==1.7.10
EOF

pip install -r requirements.txt -r requirements-dev.txt
```

Tekshirish: `python -c "import django, channels, rest_framework; print('OK')"`

---

4. NODE PAKETLARI (3D optimizatsiya)

```
cd /home/user/armenu
npm init -y
npm install --save-dev @gltf-transform/cli
```

Tekshirish: `npx gltf-transform --version`

---

5. MA'LUMOTLAR BAZASI

```
sudo -u postgres psql <<'EOF'
CREATE DATABASE armenu_db;
CREATE USER armenu WITH PASSWORD 'CHANGE_ME_STRONG_PASSWORD';
ALTER ROLE armenu SET client_encoding TO 'utf8';
ALTER ROLE armenu SET default_transaction_isolation TO 'read committed';
ALTER ROLE armenu SET timezone TO 'Asia/Tashkent';
GRANT ALL PRIVILEGES ON DATABASE armenu_db TO armenu;
\c armenu_db
GRANT ALL ON SCHEMA public TO armenu;
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS btree_gin;
EOF
```

Tekshirish:

```
PGPASSWORD='CHANGE_ME_STRONG_PASSWORD' psql -h localhost -U armenu -d armenu_db \
  -c "SELECT extname FROM pg_extension;"
# vector, pg_trgm, btree_gin ko'rinishi kerak
```

---

6. REDIS

⚠ **`db=5` ishlatiladi** — mebelcity `db=0..4` ni band qilgan.

```
redis-cli -n 5 SET armenu_test "ok"
redis-cli -n 5 GET armenu_test
redis-cli -n 5 DEL armenu_test
```

Tekshirish: `ok` chiqishi kerak.

---

7. .env FAYL

```
cd /home/user/armenu
cat > .env <<'EOF'
# --- Asosiy ---
DEBUG=True
SECRET_KEY=CHANGE_ME_GENERATE_RANDOM_50_CHARS
ALLOWED_HOSTS=localhost,127.0.0.1
PUBLIC_DOMAIN=localhost:8060

# --- Ma'lumotlar bazasi ---
DB_NAME=armenu_db
DB_USER=armenu
DB_PASSWORD=CHANGE_ME_STRONG_PASSWORD
DB_HOST=127.0.0.1
DB_PORT=5432

# --- Redis (⚠ db=5) ---
REDIS_URL=redis://127.0.0.1:6379/5
REDIS_CHANNEL_URL=redis://127.0.0.1:6379/6

# --- AI provayderlar (kamida 2 tasi to'ldirilsin) ---
ANTHROPIC_API_KEY=
GEMINI_API_KEY=
OPENAI_API_KEY=
GROQ_API_KEY=
DEEPSEEK_API_KEY=
MISTRAL_API_KEY=

# --- 3D generatsiya ---
MESHY_API_KEY=
TRIPO_API_KEY=

# --- AI limitlari ---
AI_DAILY_LIMIT_PER_RESTAURANT=2.0
AI_MONTHLY_LIMIT_TOTAL=500.0

# --- To'lov (FAZA 10 da) ---
PAYME_MERCHANT_ID=
PAYME_KEY=
CLICK_SERVICE_ID=
CLICK_SECRET=
UZUM_MERCHANT_ID=
UZUM_SECRET=

# --- Fiskal (FAZA 10 da) ---
OFD_PROVIDER=
OFD_LOGIN=
OFD_PASSWORD=

# --- Telegram ---
TELEGRAM_BOT_TOKEN=

# --- Sentry (ixtiyoriy) ---
SENTRY_DSN=
EOF

cat > .gitignore <<'EOF'
venv/
__pycache__/
*.pyc
.env
*.sqlite3
media/
staticfiles/
node_modules/
.pytest_cache/
htmlcov/
.coverage
*.log
.DS_Store
EOF
```

⚠ `SECRET_KEY` **generatsiya:**

```
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

Natijani `.env` ga qo'y.

Tekshirish: `grep -c "=" .env` → 30 dan ko'p

---

8. DJANGO SKELET

```
cd /home/user/armenu
django-admin startproject config .
mkdir -p core apps static templates locale tests scripts deploy
touch core/__init__.py apps/__init__.py
```

Tekshirish: `ls` → `config/ core/ apps/ manage.py` bor

---

9. PYTEST SOZLAMASI

```
cat > pytest.ini <<'EOF'
[pytest]
DJANGO_SETTINGS_MODULE = config.settings.test
python_files = test_*.py
addopts = --reuse-db --nomigrations -q
testpaths = tests
EOF
```

---

10. MAKEFILE (qulaylik)

```
cat > Makefile <<'EOF'
.PHONY: run ws worker test lint migrate shell

run:
    python manage.py runserver 0.0.0.0:8060

ws:
    daphne -b 0.0.0.0 -p 8061 config.asgi:application

worker:
    python manage.py run_worker --queue=$(Q)

test:
    pytest -x -q

test-cov:
    pytest --cov=apps --cov=core --cov-report=term-missing

lint:
    ruff check . && black --check .

fmt:
    ruff check --fix . && black .

migrate:
    python manage.py makemigrations && python manage.py migrate

shell:
    python manage.py shell

seed:
    python manage.py loaddata scripts/demo.json
EOF
```

---

11. BIRINCHI COMMIT

```
cd /home/user/armenu
git add -A
git commit -m "V-001: loyiha skeleti, muhit tayyorlandi"
```

✅ YAKUNIY TEKSHIRUV

Quyidagi skriptni ishga tushir — hammasi ✅ bo'lishi kerak:

```
cat > scripts/check_env.sh <<'EOF'
#!/bin/bash
ok(){ echo "✅ $1"; }
no(){ echo "❌ $1"; FAIL=1; }

[ -d venv ] && ok "venv" || no "venv yo'q"
python -c "import django" 2>/dev/null && ok "Django" || no "Django yo'q"
python -c "import channels" 2>/dev/null && ok "Channels" || no "Channels yo'q"
python -c "import pgvector" 2>/dev/null && ok "pgvector" || no "pgvector yo'q"
[ -f .env ] && ok ".env" || no ".env yo'q"
[ -f manage.py ] && ok "manage.py" || no "manage.py yo'q"
redis-cli -n 5 PING >/dev/null 2>&1 && ok "Redis db=5" || no "Redis yo'q"
PGPASSWORD=$(grep DB_PASSWORD .env|cut -d= -f2) psql -h 127.0.0.1 -U armenu \
  -d armenu_db -c "SELECT 1" >/dev/null 2>&1 && ok "PostgreSQL" || no "PostgreSQL yo'q"
npx gltf-transform --version >/dev/null 2>&1 && ok "gltf-transform" || no "gltf-transform yo'q"

[ -z "$FAIL" ] && echo -e "\n🎉 MUHIT TAYYOR — FAZA 0 ga o'ting" || echo -e "\n⚠ Xatolarni tuzating"
EOF
chmod +x scripts/check_env.sh
./scripts/check_env.sh
```

🎯 KEYINGI QADAM

```
cat ROADMAP/01_FAZALAR/FAZA_00_TAYYORGARLIK.md
```

Va `PROGRESS.md` da `V-001` ni belgilash:

```
python3 ROADMAP/03_HOLAT/update_progress.py V-001 ok
```

06

Ma'lumotnoma indeksi

> **Maqsad:** Butun faylni o'qimasdan, aynan kerakli bo'limni topish.
> `TZ_MASTER/` fayllari jami 260 KB (~70 000 token). To'liq o'qish — **isrof**.
> Bu indeks orqali 2-5 KB o'qib, javobni topasan.

⚡ QANDAY ISHLATILADI

```
# 1. Indeksdan bo'lim raqamini top
grep -i "izlagan_narsang" ROADMAP/02_MALUMOTNOMA/INDEKS.md

# 2. Faqat o'sha bo'limni o'qi
sed -n '340,420p' "TZ_MASTER/04_DATA_MODEL_VA_API.md"

# yoki
grep -A 45 "class Order" "TZ_MASTER/04_DATA_MODEL_VA_API.md"
```

📄 FAYL 01 — `TZ_MASTER/01_KIRISH_VA_BIZNES.md` (24 KB)

Qachon kerak: biznes qarori, narx, bozor raqami, investor savoli

§	Mavzu	Kalit so'zlar
1	Hujjat haqida	kim o'qiydi, printsip
2	Loyiha mohiyati	nima quriladi, nima uchun ishlaydi
3	⛔ Izolyatsiya kafolati	mebelcity, port, DB, servis
4	Biznes-model	daromad oqimi, **tarif narxlari**, birlik iqtisodiyoti, LTV/CAC
4.4	**6/12/24 oylik maqsad**	viloyat, respublika, qo'shni davlat
4.5	**5 yil himoya devorlari**	moat, kutubxona, platforma effekti
5	Bozor raqamlari	27312 korxona, turizm, TAM/SAM/SOM
6	Raqobat matritsasi	Dastur-xon, Zoomda, iiko, pozitsiyalash
7	Sotuv strategiyasi	pitch, kanallar, CAC
8	Investor	investitsiya, metrika, exit
9	Restoran egasi	"men nima olaman", xavotirlar
10	Davlat	fiskalizatsiya, qonun
11	Foydalanuvchi	mijoz hikoyasi, turist

📄 FAYL 02 — `TZ_MASTER/02_SAHIFALAR_VA_TUGMALAR.md` (68 KB) ⭐ eng ko'p kerak

Qachon kerak: "bu tugma nima qiladi", "bu sahifada nima bor"

Mijoz sahifalari

Kod	Sahifa	Qidiruv
M-01	QR kirish (redirect)	`grep -A 60 "## M-01"`
M-02	Menyu (bosh sahifa)	`grep -A 70 "## M-02"`
M-04	Taom kartochkasi (modal)	`grep -A 60 "## M-04"`
M-05	⚡ **AR ko'rish**	`grep -A 90 "## M-05"`
M-06	Savat + submitOrder()	`grep -A 80 "## M-06"`
M-08	Buyurtma holati (real-time)	`grep -A 40 "## M-08"`
M-09	Chaqiruv	`grep -A 25 "## M-09"`
M-10	Hisob va to'lov	`grep -A 70 "## M-10"`
M-11	Choychaqa (tips)	`grep -A 25 "## M-11"`
M-12	Otziv + Google filtri	`grep -A 45 "## M-12"`
M-13	AI chatbot	`grep -A 55 "## M-13"`

Ofitsiant

Kod	Sahifa
O-01	PIN kirish
O-02	Bosh ekran (yangi buyurtmalar)
O-05	2D zal plani

KDS / Admin / Super-admin

Kod	Sahifa
K-01	Oshxona monitori (kanban)
A-01	Dashboard
A-04	⚡ 3D model boshqaruvi + AI yaratish zanjiri
A-05	Stollar va QR
A-10	AR analitika
A-15	Tarjima boshqaruvi
P-02	3D kutubxona (super-admin)
P-04	Tizim salomatligi

📄 FAYL 03 — `TZ_MASTER/03_ARXITEKTURA.md` (36 KB)

Qachon kerak: masshtablash, xato, tiklanish, xavfsizlik

§	Mavzu	Kalit so'zlar
1	Arxitektura printsiplari	stateless, idempotency, circuit breaker
2	Umumiy arxitektura	CDN, LB, web, WS, DB, worker
3	Modullar	Django app'lar, bog'liqlik
4	**Masshtablash**	S0…S5 bosqichlar
4.2	⚡ **1 mln bir vaqtda**	WS optimizatsiya, DB, narx
4.3	Kod qoidalari	N+1, iterator, cache
5	**Xatolarga chidamlilik**	yiqilish matritsasi
5.2	Circuit breaker (kod)	CLOSED/OPEN/HALF_OPEN
5.3	Retry siyosati	urinish, kechikish
5.4	Graceful degradation	fallback zanjirlari
6	**Self-healing**	15 mexanizm, daemon kodi
6.3	Chaos engineering	atayin yiqitish
7	SLA/SLO	javob vaqti, mavjudlik, RPO/RTO
8	Xavfsizlik	qatlamlar, tenant izolyatsiya, rate limit
9	Kuzatuv	Prometheus metrikalar, alertlar
10	Backup	chastota, tiklash

📄 FAYL 04 — `TZ_MASTER/04_DATA_MODEL_VA_API.md` (36 KB) ⭐ eng ko'p kerak

Qachon kerak: model maydonlari, API format, WS xabar

Modellar (§1)

Model	Qidiruv
Restaurant	`grep -A 55 "class Restaurant"`
Hall, Table	`grep -A 45 "class Table"`
Category, Dish	`grep -A 60 "class Dish"`
DishLibrary ⚡	`grep -A 40 "class DishLibrary"`
DishModel3D	`grep -A 25 "class DishModel3D"`
ARView	`grep -A 20 "class ARView"`
Order	`grep -A 45 "class Order"`
Holat mashinasi	`grep -A 30 "ALLOWED = {"`
Qolgan jadvallar	`grep -A 20 "Qolgan jadvallar"`

API (§2)

Nima	Qidiruv
Umumiy qoidalar	`grep -A 15 "2.1 Umumiy qoidalar"`
Xato formati + kodlar	`grep -A 30 "2.2 Xato formati"`
Endpoint ro'yxati (to'liq)	`grep -A 80 "2.3 Endpointlar"`
GET /v1/menu/{slug}/	`grep -A 55 "GET /v1/menu"`
POST /v1/orders/	`grep -A 90 "POST /v1/orders/"`
create_order() to'liq kod	`grep -A 70 "def create_order"`

WebSocket (§3)

Nima	Qidiruv
Ulanish nuqtalari	`grep -A 12 "3.1 Ulanish"`
Xabar turlari	`grep -A 25 "3.3 Xabar turlari"`
ReliableWS klient	`grep -A 40 "class ReliableWS"`

📄 FAYL 05 — `TZ_MASTER/05_KOD_YOZISH_TARTIBI.md` (28 KB)

Qachon kerak: fayl strukturasi, vazifa ro'yxati

> ⚠ Bu fayl **eskirgan** — vazifalar endi `ROADMAP/01_FAZALAR/` da to'liq yozilgan.
> Faqat **fayl strukturasi** (§0) uchun kerak bo'lishi mumkin.

§	Mavzu
0	**Fayl strukturasi** (to'liq daraxt)
1-12	Vazifalar ro'yxati (→ FAZALAR papkasiga ko'chirilgan)

📄 FAYL 06 — `TZ_MASTER/06_TARJIMA_VA_UI.md` (24 KB)

Qachon kerak: tarjima, rang, shrift, komponent

§	Mavzu	Kalit so'zlar
1	3 tilli tizim	qamrov, 3 qatlam
1.3	⚡ **i18n JSON tuzilmasi**	to'liq lug'at namunasi
1.4	JS t() funksiyasi	loadI18n, applyI18n, switchLang
1.5	CI tekshiruvi	kalitlar mos, bo'sh yo'q
1.6	AI avto-tarjima	prompt, servis
2.2	⚡ **Rang tizimi (CSS)**	`:root` , dark, light
2.3	Tipografiya	shrift, o'lchamlar
2.4	KDS uslubi	neon, katta shrift
2.5	Komponentlar	button, card, modal, toast
2.6	Animatsiyalar	davomiylik
2.8	Ochiqlik	kontrast, ARIA
3	PWA	manifest, service worker, offline

📄 FAYL 07 — `TZ_MASTER/07_AI_MCP.md` (28 KB)

Qachon kerak: AI provayder, chatbot, tarjima

§	Mavzu	Kalit so'zlar
2	MCP arxitekturasi	diagramma
3	**8 provayder ro'yxati**	narx, env kalit, vazifalar
4.1	MCPProvider baza sinf	ABC, Request, Response
4.2	Claude provayder (kod)	namuna
4.3	Meshy provayder (kod)	3D, poll
4.4	⚡ **MCPRouter (kod)**	fallback, kesh, byudjet
4.5	Ishlatilishi	`mcp.run(...)`
5	API key boshqaruvi	.env, xavfsizlik
6	AI vazifalari + xarajat	oylik $
7	**Chatbot**	system prompt, servis, kontekst
8	Testlar	fallback, byudjet
9	AI kuzatuvi	P-05 sahifasi

📄 FAYL 08 — `TZ_MASTER/08_DEPLOY_REJA_RISK.md` (20 KB)

Qachon kerak: deploy, konfig, risk, savol

§	Mavzu
1.2	⚡ **systemd servislar** (to'liq konfig)
1.3	**nginx konfig** (to'liq)
1.4	CI/CD (GitHub Actions)
2	To'liq reja (hafta-hafta)
2.4	6/12/24 oylik yo'l xaritasi
3	Jamoa
4	**Risk matritsasi**
5	Sifat nazorati, test qamrovi
6	⚡ **Qabul mezonlari (v1)**
7	12 ta ochiq savol

🎯 TEZ JAVOBLAR (eng ko'p so'raladigan)

Savol	Javob
Port qaysi?	web **8060**, ws **8061**
Redis DB?	`db=5` (cache), `db=6` (channels)
DB nomi?	`armenu_db`
GLB maks hajm?	**3 MB**
USDZ maks hajm?	**5 MB**
AR o'lcham atributi?	`ar-scale="fixed"` + model **metrda**
WS guruh nomi?	`rest_<id>_kds`, `rest_<id>_waiters`, `guest_<sid>`
Tarif narxlari?	FREE 0 / START 199k / PRO 590k / FISKAL 890k
Test qamrovi?	≥75% umumiy, ≥90% orders/payments/tenants
Buyurtma holatlari?	new→accepted→cooking→ready→served→paid
Tillar?	uz, ru, en
AI provayderlar?	Claude, Gemini, OpenAI, Groq, DeepSeek, Mistral, Meshy, Tripo
Chaqiruv cooldown?	60 soniya
Idempotency TTL?	24 soat
Menyu kesh TTL?	60 soniya

07

Tez yordam

> Bu faylni ochiq tut. Har vazifada takrorlanadigan naqshlar shu yerda.
> Boshqa faylga qarash shart emas.

---

1. YANGI MODEL YARATISH

```
# apps/<app>/models.py
from django.db import models
from core.models import TenantModel        # ⚡ restaurant FK bor

class Yangi(TenantModel):
    name_uz = models.CharField(max_length=200)
    name_ru = models.CharField(max_length=200, blank=True)
    name_en = models.CharField(max_length=200, blank=True)
    is_active = models.BooleanField(default=True, db_index=True)

    class Meta:
        indexes = [models.Index(fields=['restaurant', 'is_active'])]

    def __str__(self):
        return f'{self.name_uz} ({self.restaurant_id})'
```

```
python manage.py makemigrations <app> && python manage.py migrate
```

---

2. YANGI VIEWSET (tenant-scoped)

```
# apps/<app>/views.py
from rest_framework.viewsets import ModelViewSet
from core.mixins import TenantScopedMixin
from core.permissions import IsRestAdmin

class YangiViewSet(TenantScopedMixin, ModelViewSet):     # ⚡ mixin SHART
    queryset = Yangi.objects.all()
    serializer_class = YangiSerializer
    permission_classes = [IsRestAdmin]
```

```
# apps/<app>/urls.py
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register('yangi', YangiViewSet, basename='yangi')
urlpatterns = router.urls
```

```
# config/urls.py ga qo'shish
path('v1/admin/', include('apps.<app>.urls')),
```

---

3. TENANT IZOLYATSIYA TESTI (har model uchun MAJBURIY)

```
# tests/test_tenant_isolation.py ga qo'shiladi
@pytest.mark.django_db
def test_yangi_tenant_isolation(api_client, restaurant, restaurant2, admin_token):
    obj = Yangi.objects.create(restaurant=restaurant2, name_uz='B ning obyekti')
    r = api_client.get(f'/v1/admin/yangi/{obj.id}/',
                       HTTP_AUTHORIZATION=f'Bearer {admin_token}')
    assert r.status_code in (403, 404)      # ❌ 200 BO'LMASLIGI SHART
```

---

4. 3 TILLI MATN

```
# Backend
from core.i18n import t
name = t(dish, 'name', lang)        # dish.name_ru → bo'sh bo'lsa name_uz
```

```
// Frontend
btn.textContent = t('dish.add_to_cart');
// yoki HTML: <button data-i18n="dish.add_to_cart"></button>
```

```
// static/i18n/uz.json, ru.json, en.json — 3 tasida ham bir xil kalit
{"dish": {"add_to_cart": "Savatga qo'shish"}}
```

---

5. DARK + LIGHT TEMA

```
/* ✗ hard-code YO'Q */
.card { background: #fff; }

/* ✓ o'zgaruvchi */
.card {
  background: var(--bg-card);
  color: var(--text);
  border: 1px solid var(--border);
}
```

Mavjud o'zgaruvchilar: `--bg --bg-soft --bg-card --border --text --text-soft --text-muted --brand --success --warning --danger --info`

---

6. WEBSOCKET XABAR YUBORISH

```
from core.ws import ws_send, g_kds, g_waiters, g_admin, g_guest

ws_send(g_kds(order.restaurant_id),     'order.new', serialize(order))
ws_send(g_waiters(order.restaurant_id), 'order.new', {'table': 5, 'id': order.id})
ws_send(g_guest(session_id),            'order.created', {'id': order.id})
```

```
// Klient
ws.onmessage = (e) => {
  const m = JSON.parse(e.data);
  if (m.type === 'order.new') { /* ... */ }
};
```

---

7. NAVBATGA QO'YISH (uzoq ish)

```
from core.queue import enqueue

# Web so'rovda — DARHOL qaytadi
enqueue('model3d', 'generate', dish_id=15, images=[url])
enqueue('fiscal',  'send_receipt', order_id=42, delay=0)
enqueue('notify',  'telegram', chat_id=..., text=..., delay=5)
```

Navbatlar: `model3d`, `fiscal`, `notify`, `ai`, `analytics`

---

8. TASHQI API (circuit breaker bilan)

```
from core.resilience import CircuitBreaker

breaker = CircuitBreaker('ofd', fail_threshold=3, timeout=120)

result = breaker.call(
    ofd_api.send, receipt,
    fallback=lambda: enqueue('fiscal', 'retry', receipt_id=receipt.id, delay=300)
)
```

---

9. KESH

```
from django.core.cache import cache
from core.cache import bump_menu_version

# O'qish
data = cache.get(key)
if data is None:
    data = expensive_query()
    cache.set(key, data, 60)

# Menyu o'zgarganda — kesh eskiradi
bump_menu_version(restaurant_id)
```

---

10. TRANZAKSIYA + POST-COMMIT

```
from django.db import transaction

with transaction.atomic():
    order = Order.objects.create(...)
    OrderItem.objects.bulk_create(items)

# ⚡ Transaksiya MUVAFFAQIYATLI bo'lgandan keyin
transaction.on_commit(lambda: ws_send(...))
transaction.on_commit(lambda: enqueue('notify', ...))
```

---

11. IDEMPOTENCY

```
from core.idempotency import check_idempotency, save_idempotency

idem_key = request.headers.get('X-Idempotency-Key')
existing = check_idempotency(idem_key)
if existing:
    return Response(existing)          # dublikat yaratilmaydi

# ... yaratish ...
save_idempotency(idem_key, result)
```

---

12. XATO CHIQARISH

```
from core.exceptions import DishUnavailable, PlanLimit, TenantMismatch

if not dish.is_available:
    raise DishUnavailable({'dish_id': dish.id, 'dish_name': dish.name_uz})
```

Javob avtomatik 3 tilda formatlanadi.

---

13. AI CHAQIRUV

```
from apps.ai.mcp import mcp, MCPRequest

resp = mcp.run(MCPRequest(
    task='chat',                    # chat|translate|vision|image_to_3d
    system=SYSTEM_PROMPT,
    prompt=question,
    max_tokens=350,
), restaurant_id=rid)

if resp.ok:
    answer = resp.content
else:
    answer = t('chat.error')        # fallback
```

---

14. MIQDOR (butun son)

```
const qty = Math.max(1, Math.min(99, Math.ceil(rawQty)));
```

```
qty = max(1, min(99, math.ceil(raw_qty)))
```

---

15. NARX HISOBI (⚡ faqat serverda)

```
# ✗ NOTO'G'RI — frontenddan kelgan summa
total = request.data['total']

# ✓ TO'G'RI — serverda qayta hisoblanadi
subtotal = sum((d.price + mod_sum) * qty for ...)
service_fee = subtotal * restaurant.service_fee_pct / 100
total = subtotal + service_fee
```

---

16. FOYDALI BUYRUQLAR

```
# Ishga tushirish
make run                    # web  :8060
make ws                     # ws   :8061
make worker Q=model3d       # worker

# Test
make test                   # tez
make test-cov               # qamrov bilan
pytest tests/test_orders.py -v -k "idempotency"

# Kod
make fmt                    # formatlash
make lint                   # tekshirish

# DB
make migrate
python manage.py shell
python manage.py dbshell

# Progress
python3 ROADMAP/03_HOLAT/update_progress.py V-043 ok
python3 ROADMAP/03_HOLAT/update_progress.py --status
```

---

17. QIDIRUV (token tejash)

```
# ✗ butun fayl (36 KB)
cat "TZ_MASTER/04_DATA_MODEL_VA_API.md"

# ✓ faqat kerakli qism (2 KB)
grep -A 45 "class Order" "TZ_MASTER/04_DATA_MODEL_VA_API.md"
grep -A 30 "ALLOWED = {" "TZ_MASTER/04_DATA_MODEL_VA_API.md"
sed -n '340,400p' "TZ_MASTER/04_DATA_MODEL_VA_API.md"

# Indeksdan topish
grep -i "order" ROADMAP/02_MALUMOTNOMA/INDEKS.md
```

---

18. TIPIK XATOLAR VA YECHIM

Xato	Sabab	Yechim
`Restoran aniqlanmadi`	`TenantMiddleware` restaurant topmadi	URL `/m/<slug>/` yoki JWT tekshir
`RelatedObjectDoesNotExist`	`select_related` yo'q	`.select_related('table')` qo'sh
WS xabar kelmayapti	Guruh nomi noto'g'ri	`rest_<id>_kds` formatini tekshir
Kesh eskirmayapti	`bump_menu_version()` chaqirilmagan	Signal qo'sh
Test `200` qaytdi	`TenantScopedMixin` yo'q	Mixin qo'sh
GLB > 3 MB	Optimizatsiya yetarli emas	`--ratio 0.3` bilan qayta
AR o'lchami noto'g'ri	`ar-scale` yoki masshtab	`ar-scale="fixed"` + metrda
Migratsiya konflikt	Parallel branch	`makemigrations --merge`

0 8

Test strategiyasi

> Testlar **kod bilan birga** yoziladi, oxirida emas.
> Har vazifa tugaganda test o'tishi shart.

---

1. TEST PIRAMIDASI



---

2. TEST TURLARI VA QACHON

Tur	Qachon yoziladi	Fayl	Tezlik
Unit	Har servis/funksiya bilan	`tests/unit/`	~1ms
Integratsiya	Har endpoint bilan	`tests/api/`	~50ms
WS	Har consumer bilan	`tests/ws/`	~100ms
E2E	Faza oxirida	`tests/e2e/`	~2s
Yuklama	Reliz oldidan	`tests/load/`	daqiqalar

---

3. ⛔ 5 TA MAJBURIY TEST (deploy bloklaydi)

CI'da bu testlar o'tmasa **deploy to'xtatiladi**:

3.1 Tenant izolyatsiya

Fayl: `tests/test_tenant_isolation.py`

```
import pytest
from django.apps import apps

# Barcha tenant modellari avtomatik topiladi
TENANT_MODELS = [
    m for m in apps.get_models()
    if any(f.name == 'restaurant' for f in m._meta.fields)
]

@pytest.mark.django_db
@pytest.mark.parametrize('model', TENANT_MODELS, ids=lambda m: m.__name__)
def test_model_has_restaurant_fk(model):
    """Har tenant modelida restaurant FK bo'lishi shart"""
    assert any(f.name == 'restaurant' for f in model._meta.fields)


@pytest.mark.django_db
def test_cross_tenant_dish_forbidden(api_client, restaurant, restaurant2, admin_token):
    from apps.menu.models import Dish, Category
    cat = Category.objects.create(restaurant=restaurant2, name_uz='B')
    dish = Dish.objects.create(category=cat, restaurant=restaurant2,
                               name_uz='B taomi', price=1000)
    r = api_client.get(f'/v1/admin/dishes/{dish.id}/',
                       HTTP_AUTHORIZATION=f'Bearer {admin_token}')
    assert r.status_code in (403, 404), "❌ Cross-tenant kirish ochiq!"


@pytest.mark.django_db
def test_cross_tenant_order_forbidden(api_client, restaurant, restaurant2, staff_token):
    ...  # xuddi shunday Order uchun


@pytest.mark.django_db
def test_qr_token_scoped(api_client, restaurant, restaurant2):
    """Boshqa restoran QR tokeni bilan kirilsa — o'sha restoran menyusi"""
    ...
```

3.2 Tarjima to'liqligi

Fayl: `tests/test_i18n_completeness.py`

```
import json, pathlib, re

BASE = pathlib.Path('static/i18n')
LANGS = ['uz', 'ru', 'en']

def flatten(d, prefix=''):
    out = {}
    for k, v in d.items():
        key = f'{prefix}.{k}' if prefix else k
        if isinstance(v, dict): out.update(flatten(v, key))
        else: out[key] = v
    return out

def load(lang):
    return flatten(json.load(open(BASE / f'{lang}.json', encoding='utf-8')))

def test_all_langs_same_keys():
    base = load('uz')
    for lang in LANGS[1:]:
        other = load(lang)
        missing = set(base) - set(other)
        extra   = set(other) - set(base)
        assert not missing, f"{lang} da yo'q: {sorted(missing)[:10]}"
        assert not extra,   f"{lang} da ortiqcha: {sorted(extra)[:10]}"

def test_no_empty_values():
    for lang in LANGS:
        empty = [k for k, v in load(lang).items() if not str(v).strip()]
        assert not empty, f"{lang} bo'sh: {empty[:10]}"

def test_placeholders_match():
    """{n}, {amount} — 3 tilda bir xil bo'lishi kerak"""
    base = load('uz')
    for lang in LANGS[1:]:
        other = load(lang)
        for k, v in base.items():
            p1 = set(re.findall(r'\{(\w+)\}', str(v)))
            p2 = set(re.findall(r'\{(\w+)\}', str(other.get(k, ''))))
            assert p1 == p2, f"{k}: uz={p1} {lang}={p2}"
```

3.3 Buyurtma holat mashinasi

Fayl: `tests/test_order_state.py`

```
import pytest
from apps.orders.state import transition, ALLOWED
from core.exceptions import InvalidTransition

@pytest.mark.django_db
def test_valid_flow(order):
    for st in ['accepted', 'cooking', 'ready', 'served', 'paid']:
        transition(order, st)
        assert order.status == st

@pytest.mark.django_db
def test_invalid_transition_rejected(order):
    with pytest.raises(InvalidTransition):
        transition(order, 'paid')          # new → paid MUMKIN EMAS

@pytest.mark.django_db
def test_paid_is_terminal(order):
    for st in ['accepted','cooking','ready','served','paid']:
        transition(order, st)
    with pytest.raises(InvalidTransition):
        transition(order, 'cancelled')

@pytest.mark.django_db
def test_timestamps_recorded(order):
    transition(order, 'accepted')
    assert order.accepted_at is not None
```

3.4 Narx hisobi (serverda)

Fayl: `tests/test_price_calculation.py`

```
@pytest.mark.django_db
def test_price_from_server_not_client(api_client, guest_session, dish):
    """Frontenddan yuborilgan narx E'TIBORGA OLINMAYDI"""
    dish.price = 50000; dish.save()
    r = api_client.post('/v1/orders/', {
        'session_id': guest_session.id, 'table_id': guest_session.table_id,
        'items': [{'dish_id': dish.id, 'qty': 2, 'price': 1}],   # ⚡ soxta narx
    }, format='json', HTTP_X_IDEMPOTENCY_KEY='k1')
    assert r.status_code == 201
    assert r.data['subtotal'] == 100000       # 50000 × 2, soxta narx emas

@pytest.mark.django_db
def test_qty_is_integer(api_client, guest_session, dish):
    r = api_client.post('/v1/orders/', {..., 'items': [{'dish_id': dish.id, 'qty': 2.7}]},
                        format='json', HTTP_X_IDEMPOTENCY_KEY='k2')
    assert r.data['items'][0]['qty'] == 3     # Math.ceil

@pytest.mark.django_db
def test_service_fee_applied(api_client, guest_session, dish, restaurant):
    restaurant.service_fee_pct = 10; restaurant.save()
    ...
    assert r.data['service_fee'] == r.data['subtotal'] * 0.10
```

3.5 To'lov idempotentligi

Fayl: `tests/test_payment_idempotency.py`

```
@pytest.mark.django_db
def test_duplicate_order_not_created(api_client, guest_session, dish):
    payload = {...}
    r1 = api_client.post('/v1/orders/', payload, format='json',
                         HTTP_X_IDEMPOTENCY_KEY='same-key')
    r2 = api_client.post('/v1/orders/', payload, format='json',
                         HTTP_X_IDEMPOTENCY_KEY='same-key')
    assert r1.data['id'] == r2.data['id']
    assert Order.objects.count() == 1

@pytest.mark.django_db
def test_webhook_processed_once(api_client, payment):
    for _ in range(3):
        api_client.post('/v1/payments/webhook/payme/', webhook_payload, format='json')
    payment.refresh_from_db()
    assert payment.status == 'paid'
    assert Payment.objects.filter(order=payment.order, status='paid').count() == 1
```

---

4. FAZA BO'YICHA TEST TALABLARI

Faza	Majburiy testlar	Qamrov
0	resilience, exceptions	60%
1	⛔ **tenant izolyatsiya**, auth, JWT, PIN blok	90%
2	menyu API, kesh, 3 til, filtr, ⛔ i18n to'liqlik	80%
3	QR noyob, PDF, grace period	80%
4	QR→menyu oqimi, savat, PWA	75%
5	⛔ **holat mashinasi**, ⛔ **narx**, ⛔ **idempotency**, WS	90%
6	AR API, fayl limiti, o'lcham, fallback, kutubxona	80%
7	zal plan saqlash, stol holati	70%
8	hisobotlar, eksport	75%
9	MCP fallback, byudjet, chatbot validatsiya	75%
10	⛔ **to'lov idempotency**, webhook imzo, fiskal retry	90%
11	keshbek, otziv filtri	75%
12	health, self-healing, yuklama	70%

---

5. FIXTURE'LAR (`tests/conftest.py`)

```
import pytest
from rest_framework.test import APIClient

@pytest.fixture
def api_client(): return APIClient()

@pytest.fixture
def restaurant(db):
    from apps.tenants.models import Restaurant
    return Restaurant.objects.create(slug='test-rest', name='Test',
        phone='998901234567', city='Toshkent', plan='pro', is_active=True)

@pytest.fixture
def restaurant2(db):
    """⚡ Cross-tenant testlar uchun"""
    from apps.tenants.models import Restaurant
    return Restaurant.objects.create(slug='test-2', name='Ikkinchi',
        phone='998901234568', city='Samarqand', plan='start', is_active=True)

@pytest.fixture
def hall(restaurant):
    from apps.tables.models import Hall
    return Hall.objects.create(restaurant=restaurant, name='1-qavat')

@pytest.fixture
def table(hall, restaurant):
    from apps.tables.models import Table
    return Table.objects.create(hall=hall, restaurant=restaurant, number='5', capacity=4)

@pytest.fixture
def category(restaurant):
    from apps.menu.models import Category
    return Category.objects.create(restaurant=restaurant, name_uz='Milliy',
                                   name_ru='Национальные', name_en='National')

@pytest.fixture
def dish(category, restaurant):
    from apps.menu.models import Dish
    return Dish.objects.create(category=category, restaurant=restaurant,
        name_uz='Osh', name_ru='Плов', name_en='Pilaf', price=45000,
        cook_min=20, is_available=True)

@pytest.fixture
def staff(restaurant, django_user_model):
    from apps.accounts.models import Staff
    from apps.accounts.services import hash_pin
    u = django_user_model.objects.create_user('waiter1', password='x')
    return Staff.objects.create(restaurant=restaurant, user=u, role='waiter',
                                pin_hash=hash_pin('1234'))

@pytest.fixture
def staff_token(staff):
    from apps.accounts.tokens import make_token
    return make_token(staff)

@pytest.fixture
def admin_token(restaurant, django_user_model):
    from apps.accounts.models import Staff
    from apps.accounts.tokens import make_token
    u = django_user_model.objects.create_user('admin1', password='x')
    s = Staff.objects.create(restaurant=restaurant, user=u, role='admin')
    return make_token(s)

@pytest.fixture
def guest_session(restaurant, table):
    from apps.guest.models import Guest, GuestSession
    g = Guest.objects.create(restaurant=restaurant, device_token='dev-1')
    return GuestSession.objects.create(guest=g, table=table, restaurant=restaurant)

@pytest.fixture
def order(restaurant, table, guest_session, dish):
    from apps.orders.models import Order, OrderItem
    o = Order.objects.create(restaurant=restaurant, table=table,
        session_id=str(guest_session.id), number='#001',
        subtotal=45000, total=45000)
    OrderItem.objects.create(order=o, dish=dish, qty=1, price=45000)
    return o
```

---

6. E2E TESTLAR (faza oxirida)

Fayl: `tests/e2e/test_full_flow.py`

```python
@pytest.mark.django_db
def test_qr_to_order_flow(api_client, table, dish):
    """QR → menyu → savat → buyurtma → KDS → ofitsiant → to'lov"""

    # 1. QR skanerlash
    r = api_client.get(f'/t/{table.qr_token}', follow=True)
    assert r.status_code == 200

    # 2. Menyu olish
    r = api_client.get(f'/v1/menu/{table.restaurant.slug}/?lang=uz')
    assert r.status_code == 200
    assert any(d['id'] == dish.id for d in r.data['dishes'])

    # 3. Buyurtma
    r = api_client.post('/v1/orders/', {...}, format='json',
                        HTTP_X_IDEMPOTENCY_KEY='e2e-1')
    assert r.status_code == 201
    order_id = r.data['id']

    # 4. Ofitsiant qabul qiladi
    r = api_client.post(f'/v1/staff/orders/{order_id}/accept/', ...)
    assert r.status_code == 200

    # 5. Oshpaz: cooking → ready
    api_client.post(f'/v1/staff/orders/{order_id}/cook/', ...)
    api_client.post(f'/v1/staff/orders/{order_id}/ready/', ...)

    # 6. Ofitsiant olib boradi
    api_client.post(f'/v1/staff/orders/{order_id}/serve/', ...)

    from apps.orders.models import Order
    assert Order.objects.get(id=order_id).status == 'served'
```

---

7. YUKLAMA TESTI (reliz oldidan)

Fayl: `tests/load/menu.js` (k6)

```javascript
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 500 },
    { duration: '5m', target: 2000 },
    { duration: '2m', target: 5000 },
    { duration: '3m', target: 0 },
  ],
  thresholds: {
    http_req_duration: ['p(95)<250'],
    http_req_failed: ['rate<0.01'],
  },
};

export default function () {
  const r = http.get(`${__ENV.BASE_URL}/v1/menu/test-rest/?lang=uz`);
  check(r, {
    'status 200': (x) => x.status === 200,
    'has dishes': (x) => JSON.parse(x.body).dishes.length > 0,
  });
  sleep(1);
}
```

```
k6 run -e BASE_URL=https://stage.armenu.uz tests/load/menu.js
```

---

8. CHAOS TESTI (oyiga 1 marta, staging'da)

Test	Buyruq	Kutilgan natija
Web node o'chirish	`systemctl stop armenu` (1 ta)	Foydalanuvchi sezmaydi
Redis o'chirish	`systemctl stop redis`	Menyu ishlaydi (DB'dan), WS uziladi va tiklanadi
DB primary o'chirish	`systemctl stop postgresql`	30-60s yozish yo'q, o'qish uzilmaydi
AI provayder bloklash	firewall	Keyingi provayderga o'tadi
Disk to'ldirish 95%	`fallocate`	Auto-cleanup ishlaydi
Worker o'ldirish	`kill -9`	Job qayta navbatga (60s)

---

9. BUYRUQLAR

```
# Tez (ishlab chiqishda)
pytest -x -q

# To'liq + qamrov
pytest --cov=apps --cov=core --cov-report=term-missing --cov-fail-under=75

# Faqat majburiy testlar (CI)
pytest tests/test_tenant_isolation.py tests/test_i18n_completeness.py \
       tests/test_order_state.py tests/test_price_calculation.py \
       tests/test_payment_idempotency.py -v

# Bitta test
pytest tests/test_orders.py::test_idempotency -v

# Sekin testlarni chetlab o'tish
pytest -m "not slow"

# Parallel (tez)
pytest -n auto
```